

多态与开闭原则的关系

- 开闭原则的定义：开闭原则指对新增功能开放，对修改原有代码关闭，新增功能时不修改已测试完成的原有代码。
- 符合开闭原则的示例：新增MySQL课程只需要新建继承自课程父类的子类，不需要修改原有课程、调用方的任何代码。
- 开闭原则的优势：修改原有代码后需要对整个代码文件重新做全量测试，不修改原有代码可以避免额外测试工作量，降低风险。
- object类的基础说明
- object类的地位：object类是Python中所有类默认的父亲类，即使写类时不声明继承object，底层也会默认继承。
- 默认继承的体现：自定义类写def方法时，代码提示会自动出现 `__str__`、`__dir__`、`__eq__` 等内置方法，就是因为默认继承了object类，这些方法都定义在object类中。
- 底层实现说明：object类等内置类型的核心实现由C语言完成，Python层面只能看到函数声明，看不到具体实现代码。
- 类属性与对象属性的区分
- 定义位置不同：对象属性定义在 `__init__` 方法中，需要加 `self.` 前缀；类属性定义在类内部、所有函数外部，直接写变量名等于值即可，不需要加self。
- 调用方式不同：对象属性必须先创建类的实例对象，才能通过对象名调用；类属性可以直接通过类名调用，不需要创建对象，也可以通过对象名调用。
- 归属不同：类属性归整个类所有，是该类所有实例对象共享的属性；对象属性归具体实例对象所有，每个对象有自己独立的对象属性。

```
In [1]: class Animal:
    """动物父类：定义所有动物的共有属性和行为"""
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def eat(self):
        print(f"{self.name} 正在吃东西 🍖")

    def sleep(self):
        print(f"{self.name} 正在睡觉 💤")

    def info(self):
        print(f"【{self.__class__.__name__}】名字：{self.name}，年龄：{self.age}")

class Cat(Animal):
    """小猫子类：继承Animal，并添加猫咪特有的行为"""
    def __init__(self, name, age, color):
```

```

        # 调用父类的初始化方法，复用 name 和 age 的赋值逻辑
        super().__init__(name, age)
        self.color = color # 子类独有的属性

    def meow(self):
        print(f"{self.name}: 喵~ 哈吉咪! 🐱")

# 重写父类的 eat 方法，体现多态
    def eat(self):
        print(f"{self.name} 正在优雅地吃小鱼干 🐟")

class Dog(Animal):
    """小狗子类：继承Animal，并添加狗狗特有的行为"""
    def __init__(self, name, age, breed):
        super().__init__(name, age)
        self.breed = breed # 子类独有的属性

    def bark(self):
        print(f"{self.name}: 汪! 大黄来啦! 🐶")

    def eat(self):
        print(f"{self.name} 正在大口啃骨头 🍖")

# ===== 测试代码 =====
if __name__ == "__main__":
    # 创建子类实例
    hajibimi = Cat("哈吉咪", 2, "橘色")
    dahuang = Dog("大黄", 3, "中华田园犬")

    # 1. 调用子类独有方法
    hajibimi.meow()
    dahuang.bark()

    # 2. 调用从父类继承的方法
    hajibimi.sleep()
    dahuang.info()

    # 3. 演示多态：同名方法，不同表现
    print("\n--- 吃饭时间 ---")
    hajibimi.eat()
    dahuang.eat()

```

哈吉咪：喵~ 哈吉咪! 🐱

大黄：汪! 大黄来啦! 🐶

哈吉咪 正在睡觉 💤

【Dog】 名字：大黄，年龄：3岁

--- 吃饭时间 ---

哈吉咪 正在优雅地吃小鱼干 🐟

大黄 正在大口啃骨头 🍖

类方法的定义与使用规则

- 类方法的标识：类方法需要使用装饰器 `@classmethod` 在方法上方进行标识。
- 参数规则不同：类方法的第一个参数约定俗成为 `cls`，代表当前类本身，不需要手动传入参数。

- 调用方式：类方法可以直接通过类名调用，不需要创建具体的实例对象，该方法为类的所有实例对象共享。

静态方法的定义与使用规则

- 静态方法的标识：静态方法需要使用装饰器 `@staticmethod` 在方法上方进行标识。
- 参数规则：静态方法不需要传入默认的 `self` 或 `cls` 参数，可以根据需求定义任意参数，也可以不定义参数。
- 使用频率：类方法和静态方法在日常开发中使用频率较低，大多用在设计模式相关场景中，日常开发用的最多的还是对象方法。

魔术方法 `new` 的作用说明

- 创建对象的执行顺序：创建类的实例对象时，会先执行 `__new__` 方法，再执行 `__init__` 方法。
- 两个方法分工：`__new__` 方法负责创建类的实例对象并返回实例，`__init__` 方法负责给创建好的对象设置对象属性。
- 开发使用规则：日常开发不需要手动重写 `__new__` 方法，只需要编写 `__init__` 设置属性即可，本次仅做原理讲解。

```
In [2]: class Undergraduate:
        """学生基类"""
        # ===== 【类属性】所有本科生共享 =====
        lab_name = "智能感知与通信实验室"      # 所属实验室（全局统一）
        stage = "本科"                          # 培养阶段标识
        skill_template = "核心课程：{"          # 统一的输出格式模板

        def __init__(self, name):
            # ===== 【实例属性】每个学生独有 =====
            self.name = name

        def show_skills(self):
            raise NotImplementedError("子类必须实现 show_skills 方法")

        def introduce(self):
            # 通过 类名.类属性 访问共享信息
            print(f"大家好，我是{self.name}，来自【{Undergraduate.lab_name}】的{self}")

class EEStudent(Undergraduate):
    # 继承父类类属性，无需重复定义 lab_name 和 stage
    def __init__(self, name):
        super().__init__(name)
        self.skills = ["模电", "数电", "信号与系统", "DSP", "嵌入式开发"]

    def show_skills(self):
        skill_str = "、".join(self.skills)
        # 通过 self.__class__.skill_template 访问类属性（支持子类覆盖）
        print(f" → {self.__class__.skill_template.format(skill_str)}")
        print(f" → 擅长：电路设计、芯片级调试、嵌入式系统开发\n")

class CSStudent(Undergraduate):
    def __init__(self, name):
        super().__init__(name)
        self.skills = ["通信原理", "信息论", "计算机网络", "数据结构"]
```

```

def show_skills(self):
    skill_str = "、".join(self.skills)
    print(f"    → {self.__class__.skill_template.format(skill_str)}")
    print(f"    → 擅长：协议分析、算法设计、网络架构搭建\n")

class MasterStudent(Undergraduate):
    # ===== 【类属性重写】研究生覆盖父类的阶段标识 =====
    stage = "硕士"

    def __init__(self, name, degree_type):
        super().__init__(name)
        self.degree_type = degree_type

    def introduce(self):
        # self.stage 会优先找到本类的 "硕士", 而非父类的 "本科"
        print(f"大家好, 我是{self.name}, 来自【{self.lab_name}】的电子信息{self.degree_type}")

class EEMaster(MasterStudent):
    # 覆盖模板: 研究生用"核心技术"替代"核心课程"
    skill_template = "核心技术: {}"

    def __init__(self, name):
        super().__init__(name, "专硕")
        self.skills = ["深度学习", "Linux开发", "模型部署", "CUDA编程"]

    def show_skills(self):
        skill_str = "、".join(self.skills)
        # 自动使用本类覆盖后的模板 "核心技术: {}"
        print(f"    → {self.__class__.skill_template.format(skill_str)}")
        print(f"    → 擅长：神经网络优化、边缘计算、Linux驱动开发\n")

# ===== 演示 =====
if __name__ == "__main__":
    students = [EEStudent("小杨"), CSStudent("小王"), EEMaster("老谭")]

    print("=" * 50)
    print(f"        {Undergraduate.lab_name} · 技能展示会")
    print("=" * 50 + "\n")

    for s in students:
        s.introduce()
        s.show_skills()

    # 验证类属性的共享特性: 修改一处, 所有未覆盖的子类同步生效
    print("--- 动态修改类属性演示 ---")
    Undergraduate.lab_name = "新一代信息技术实验室"
    print(f"小杨的新介绍: ", end="")
    students[0].introduce() # 自动使用新实验室名
    print(f"老谭的新介绍: ", end="")
    students[2].introduce() # 同样自动更新 (因为MasterStudent未覆盖lab_name)

```

智能感知与通信实验室 · 技能展示会

大家好，我是小杨，来自【智能感知与通信实验室】的本科生 🙋

→ 核心课程：模电、数电、信号与系统、DSP、嵌入式开发
→ 擅长：电路设计、芯片级调试、嵌入式系统开发

大家好，我是小王，来自【智能感知与通信实验室】的本科生 🙋

→ 核心课程：通信原理、信息论、计算机网络、数据结构
→ 擅长：协议分析、算法设计、网络架构搭建

大家好，我是老谭，来自【智能感知与通信实验室】的电子信息专硕硕士生 🎓

→ 核心技术：深度学习、Linux开发、模型部署、CUDA编程
→ 擅长：神经网络优化、边缘计算、Linux驱动开发

--- 动态修改类属性演示 ---

小杨的新介绍：大家好，我是小杨，来自【新一代信息技术实验室】的本科生 🙋

老谭的新介绍：大家好，我是老谭，来自【新一代信息技术实验室】的电子信息专硕硕士生 🎓

```
In [3]: class Undergraduate:
    """学生基类"""
    # ===== 类属性 =====
    lab_name = "智能感知与通信实验室"
    stage = "本科"
    skill_template = "核心课程：{"}"

    # 【类属性】全局花名册，存储所有通过工厂方法创建的学生实例
    _registry = []

    def __init__(self, name):
        self.name = name

    # ===== 【类方法】工厂模式：统一创建并自动注册 =====
    @classmethod
    def create_student(cls, name, **kwargs):
        """
        统一的学生创建入口
        cls 参数代表调用该方法的实际子类（如 EEStudent、EEMaster）
        """
        # cls(name, **kwargs) 等价于 EEStudent(name) 或 EEMaster(name)
        student = cls(name, **kwargs)

        # 自动将新创建的学生加入全局花名册
        cls._registry.append(student)
        print(f"✅ [{cls.__name__}] {name} 已成功注册到 {cls.lab_name}")
        return student

    # ===== 【类方法】全局查询：按阶段筛选学生 =====
    @classmethod
    def get_students_by_stage(cls, target_stage):
        """从全局花名册中筛选指定培养阶段的学生"""
        result = [s for s in cls._registry if getattr(s, 'stage', None) == target_stage
                  or (hasattr(s, '__class__') and s.__class__.stage == target_stage)]
        return result

    def show_skills(self):
        raise NotImplementedError("子类必须实现 show_skills 方法")
```

```

def introduce(self):
    print(f"大家好，我是{self.name}，来自【{Undergraduate.lab_name}】的{self.name}")

class EEStudent(Undergraduate):
    def __init__(self, name):
        super().__init__(name)
        self.skills = ["模电", "数电", "信号与系统", "DSP", "嵌入式开发"]

    def show_skills(self):
        skill_str = ",".join(self.skills)
        print(f"    → {self.__class__.skill_template.format(skill_str)}")
        print(f"    → 擅长：电路设计、芯片级调试、嵌入式系统开发\n")

class CSStudent(Undergraduate):
    def __init__(self, name):
        super().__init__(name)
        self.skills = ["通信原理", "信息论", "计算机网络", "数据结构"]

    def show_skills(self):
        skill_str = ",".join(self.skills)
        print(f"    → {self.__class__.skill_template.format(skill_str)}")
        print(f"    → 擅长：协议分析、算法设计、网络架构搭建\n")

class MasterStudent(Undergraduate):
    stage = "硕士"

    def __init__(self, name, degree_type):
        super().__init__(name)
        self.degree_type = degree_type

    def introduce(self):
        print(f"大家好，我是{self.name}，来自【{self.lab_name}】的电子信息{self.name}")

class EEMaster(MasterStudent):
    skill_template = "核心技术：{"

    def __init__(self, name):
        super().__init__(name, "专硕")
        self.skills = ["深度学习", "Linux开发", "模型部署", "CUDA编程"]

    def show_skills(self):
        skill_str = ",".join(self.skills)
        print(f"    → {self.__class__.skill_template.format(skill_str)}")
        print(f"    → 擅长：神经网络优化、边缘计算、Linux驱动开发\n")

# ===== 演示类方法的威力 =====
if __name__ == "__main__":
    print("=" * 50)
    print("    使用类方法（工厂模式）统一创建学生")
    print("=" * 50)

    # 【核心亮点】不再直接调用 EEStudent()，而是通过基类类方法创建
    # cls 会自动绑定为对应的子类，实现多态创建
    xiao_yang = EEStudent.create_student("小杨")
    xiao_wang = CSStudent.create_student("小王")

```

```

lao_tan    = EEMaster.create_student("老谭")

print("\n" + "=" * 50)
print("    使用类方法进行全局查询")
print("=" * 50)

# 直接通过类方法查询，无需手动维护外部列表
masters = Undergraduate.get_students_by_stage("硕士")
print(f"\n🔍 实验室共有 {len(masters)} 名硕士生:")
for m in masters:
    print(f"    - {m.name} ({m.degree_type})")

undergrads = Undergraduate.get_students_by_stage("本科")
print(f"\n🔍 实验室共有 {len(undergrads)} 名本科生:")
for u in undergrads:
    print(f"    - {u.name}")

```

=====

使用类方法（工厂模式）统一创建学生

=====

- ✅ [EEStudent] 小杨 已成功注册到 智能感知与通信实验室
- ✅ [CSStudent] 小王 已成功注册到 智能感知与通信实验室
- ✅ [EEMaster] 老谭 已成功注册到 智能感知与通信实验室

=====

使用类方法进行全局查询

=====

- 🔍 实验室共有 1 名硕士生：
 - 老谭（专硕）
- 🔍 实验室共有 2 名本科生：
 - 小杨
 - 小王

模块导入规则讲解

- 模块导入的基础写法：同一目录下的Python文件可以直接通过 `import 文件名` 的方式完成模块导入，导入后会默认执行被导入文件的全部代码。
- `__name__` 变量的规则：直接运行的Python文件，其 `__name__` 变量的值固定为 `__main__`；被作为模块导入的Python文件，其 `__name__` 变量的值为自身的文件名。
- 模块导入的规范写法：将类、函数的声明放在文件顶部，所有函数调用、对象创建、测试代码都必须包裹在 `if __name__ == '__main__':` 判断内。
- 文件名命名规则：如果Python文件需要被其他文件导入，文件名不能使用中文，必须使用英文命名，否则会导入报错。

Python操作txt文件的步骤讲解

- 操作txt文件的固定流程：Python操作txt文件分为固定三步，第一步打开文件，第二步读取或写入内容，第三步关闭文件。
- `open`函数的第一个参数（file）：该参数用于指定要打开文件的路径，路径分为相对路径和绝对路径两种，相对路径是相对于当前运行的py文件的路径，绝对路径是从盘符开始的完整路径。

- open函数的第二个参数 (mode)：该参数用于指定打开文件的模式，常用模式有三种，r代表只读，文件不存在会报错；w代表只写，从头开始覆盖写入，文件不存在会自动新建；a代表追加写入，在原有文件末尾添加内容，文件不存在会自动新建。
- open函数的第三个参数 (encoding)：该参数用于指定文件的编码格式，包含中文的文件统一使用utf-8编码，如果编码格式不匹配会导致中文乱码，甚至内容损坏无法恢复。

In []: #作业

```
class MyList(list):
    def sort(self, reverse=False):
        """
        重写原生 list.sort() 方法
        :param reverse: False为升序(默认), True为降序
        """
        n = len(self)

        # 核心: 冒泡排序逻辑
        for i in range(n - 1):
            swapped = False # 优化标志: 如果本轮没交换, 说明已经有序

            for j in range(n - 1 - i):
                # 【关键】根据 reverse 参数决定比较方向
                # 升序时: 前 > 后 则交换
                # 降序时: 前 < 后 则交换
                should_swap = (self[j] > self[j + 1]) if not reverse else (self[j] < self[j + 1])

                if should_swap:
                    # 原地交换元素
                    self[j], self[j + 1] = self[j + 1], self[j]
                    swapped = True

            # 如果本轮没有发生任何交换, 提前结束排序
            if not swapped:
                break

        # 原生 sort() 返回 None, 这里也保持一致
        return None

if __name__ == '__main__':
    # 测试1: 默认升序
    lst1 = MyList([64, 34, 25, 12, 22, 11, 90])
    lst1.sort()
    print(f"升序结果: {lst1}")

    # 测试2: 降序排序
    lst2 = MyList([64, 34, 25, 12, 22, 11, 90])
    lst2.sort(reverse=True)
    print(f"降序结果: {lst2}")

    # 测试3: 验证返回值是否为 None
    result = lst1.sort()
    print(f"返回值: {result}")

    # 测试4: 空列表与单元素列表 (边界情况)
    lst3 = MyList([])
    lst3.sort()
```



```
print(f"空列表: {lst3}")

lst4 = MyList([1])
lst4.sort(reverse=True)
print(f"单元素: {lst4}")
```

升序结果: [11, 12, 22, 25, 34, 64, 90]

降序结果: [90, 64, 34, 25, 22, 12, 11]

返回值: None

空列表: []

单元素: [1]